Data 37711: Homework 2

Zien Guo

Code: `https://github.com/zienguo-SilverHand/DATA37711` (directory `homework_2`; run `pip install -r requirements.txt && bash run_all.sh`). Notebooks `hw2_q1`, `hw2_q2`, `hw2_q3` correspond to Problems 1–3.

Problem 1

(a) Let $g(x) = (g_1(x), g_2(x))$. Take

$$f_\mu(x) = g_1(x), \qquad f_\sigma(x) = \log\bigl(1 + e^{g_2(x)}\bigr).$$

The mean is unconstrained, while the softplus maps $\mathbb{R} \to (0, \infty)$, so $f_\sigma > 0$.

(b) With $Y \mid X \sim \mathcal{N}(f_\mu(X), f_\sigma(X)^2)$, the density of one observation is

$$p(y \mid x) = \frac{1}{\sqrt{2\pi}\, f_\sigma(x)} \exp\!\left(-\frac{(y - f_\mu(x))^2}{2 f_\sigma(x)^2}\right).$$

Maximum likelihood chooses θ to maximize $\prod_{i=1}^n p(y_i \mid x_i)$, which is equivalent to maximizing $\sum_i \log p(y_i \mid x_i)$, or minimizing the negative log-likelihood:

$$-\sum_{i=1}^n \log p(y_i \mid x_i) = \sum_{i=1}^n \left[\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} + \log f_\sigma(x_i) + \tfrac{1}{2}\log 2\pi\right].$$

Dropping the constant and averaging gives the loss

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^n \left[\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} + \log f_\sigma(x_i)\right].$$

(c) This part is implemented in `hw2_q1.ipynb` (70/15/15 train/validation/test split, MLP with early stopping on validation loss). Baselines are the training mean, linear regression and gradient boosting (scikit-learn); each is paired with a Gaussian with constant σ equal to its training residual standard deviation to compute NLL. Test results:

Model	RMSE (min)	MAE (min)	NLL
Mean	7.688	5.664	3.459
Linear regression	7.473	5.554	3.430
Gradient boosting	4.285	2.982	2.874
Gaussian MLP	**3.898**	**2.454**	**2.621**

The MLP beats all baselines on every metric.

(d) For an input x, let $\hat{\mu}(x) = f_{\hat{\mu}}(x)$ and $\hat{\sigma}(x) = f_{\hat{\sigma}}(x)$ be the fitted model's outputs. Then $Y \mid x \sim \mathcal{N}(\hat{\mu}(x), \hat{\sigma}(x)^2)$ and

$$\hat{P}(Y < 45 \mid x) = P\left(\frac{Y - \hat{\mu}(x)}{\hat{\sigma}(x)} < \frac{45 - \hat{\mu}(x)}{\hat{\sigma}(x)}\right) = \Phi\left(\frac{45 - \hat{\mu}(x)}{\hat{\sigma}(x)}\right),$$

where Φ is the standard normal CDF.

(e) I computed $\hat{p}_i = \Phi\big((45 - \hat{\mu}(x_i))/\hat{\sigma}(x_i)\big)$ on the test set and compared with $\mathbf{1}\{y_i < 45\}$. For accuracy, I threshold $\hat{p}_i$ at 0.5. The baselines use the same formula with their constant σ.

Model	Brier score	Log-loss	Accuracy
Mean	0.0043	0.0532	0.9957
Linear regression	0.0043	0.0445	0.9957
Gradient boosting	0.0031	0.0318	**0.9967**
Gaussian MLP	**0.0030**	**0.0213**	0.9965

Since 99.57% of test trips are under 45 minutes (only 0.43% take 45 minutes or longer), predicting "under 45" for every trip already gives accuracy 0.9957, so accuracy barely separates the models. The MLP has the best Brier score and log-loss.

Problem 2

(a) `StockCode` has 3,665 unique values and `CustomerID` has 4,339, so one-hot encoding needs 8,004 columns for 397,924 rows. It is possible to store sparsely, but the input is very high-dimensional and many categories are rare, so target encoding or embeddings are better choices. `Quantity` is heavy-tailed (median 6, maximum 80,995), so I model $\log(1 + \texttt{Quantity})$ with MSE. The data is split 70/15/15% (validation used for early stopping). All errors below are on the $\log(1 + \cdot)$ scale.

(b)–(d) Implemented in `hw2_q2.ipynb`. All three models use the same MLP (two hidden layers of 128) and differ only in inputs: (b) the two target-encoded features (scikit-learn `TargetEncoder`; training-set encodings use its 5-fold cross-fitted `fit_transform`, so a row's own label does not enter its encoding, and validation/test rows use the encoder fit on the full training set); (c) a 16-dimensional embedding for each feature; (d) both. Test results:

Model	Test RMSE	Test MAE
Mean baseline	1.0072	0.8193
(b) Target encoding	0.5797	0.4263
(c) Embeddings	0.5474	0.3905
(d) Target encoding + embeddings	**0.5369**	**0.3799**

Embeddings beat target encoding, and combining the two is slightly better again.

(e) Target encoding compresses each category into a single number (its mean target), which is noisy for rare categories and ignores the other feature. An embedding layer learns a vector per category jointly with the network, so it can store whatever helps prediction, and the network can combine the `StockCode` and `CustomerID` vectors nonlinearly to capture product–customer interactions. This is consistent with the test results above. The combined model adds a small gain, presumably because the target-encoded values give a useful summary for rare categories. Almost all test rows (59,635 of 59,689) have both categories seen in training; the few rows with unseen categories (30 and 24) are too few to draw conclusions.

Problem 3

(a) Figure 1 shows the training data. The response is smooth and nonlinear: lowest (about -1) near $(x_1, x_2) \approx (-1.5, 0)$, and highest (about 5) near the top and bottom edges at $x_1 \approx 1$ to 2, like a noisy saddle.

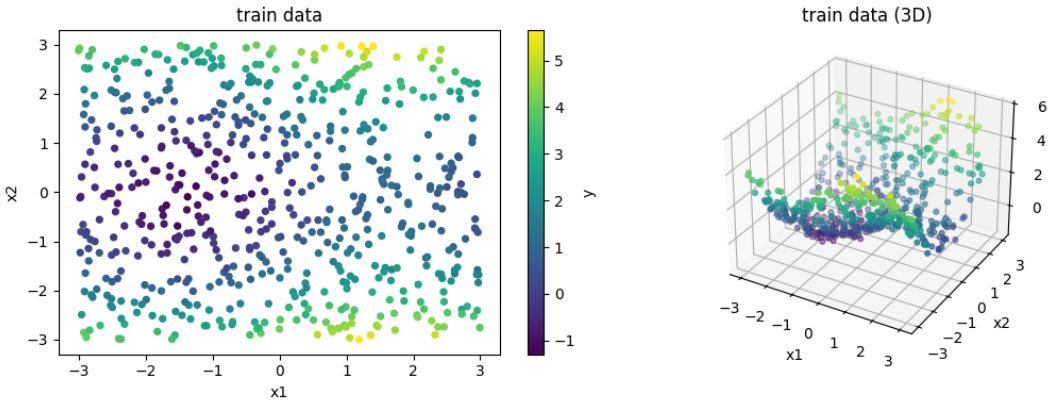


Figure 1: Training data.

(b) RKS with $D = 500$ sine/cosine features and ridge penalty $\lambda = 10^{-3}$ (fixed); I swept σ over 20 log-spaced values in $[10^{-1.5}, 10]$ (Figure 2). Small σ (high-frequency features) overfits badly, while for $\sigma \gtrsim 0.3$ the validation MSE is flat and small. The best value is $\sigma = 5.46$ (validation MSE 0.0362).

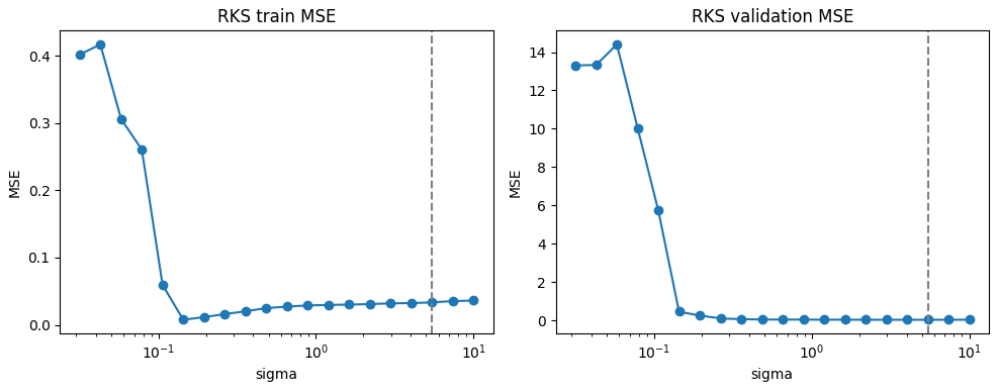


Figure 2: RKS training (left) and validation (right) MSE vs. σ; dashed line is the best σ.

(c) One-hidden-layer ReLU MLP (Adam, lr 10^{-2}, batch 64, 300 epochs); I swept the hidden dimension over $\{2, \ldots, 512\}$ (Figure 3). Widths 2 and 4 underfit; the best is 64 (validation MSE 0.0403), and the widest networks are slightly worse.

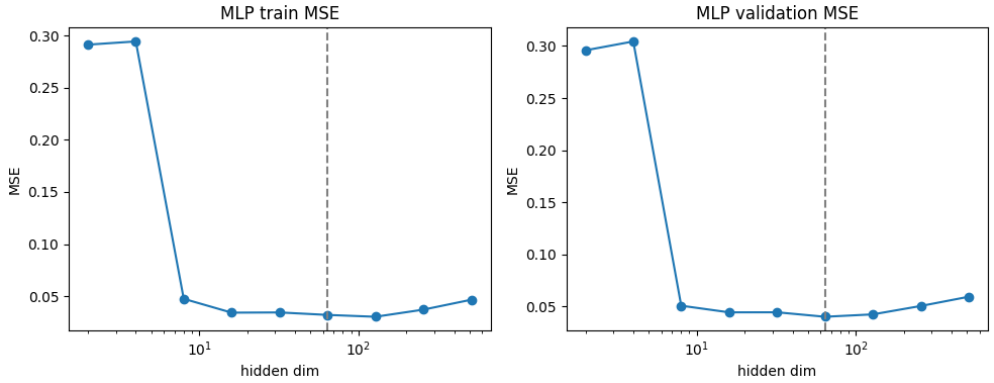


Figure 3: MLP training (left) and validation (right) MSE vs. hidden dimension.

(d) After refitting on the training set only (same seeds, so the validation MSEs equal the sweep values) with the selected hyperparameters, the validation MSE is 0.0362 for RKS and 0.0403 for the MLP. RKS is slightly better, but the gap is small and comes from a single seed, so I do not claim a clear winner. The test predictions are in `rks_test_predictions.csv` and `mlp_test_predictions.csv`.

Figure 4 compares the fitted surfaces with the data. Both capture the basin near $(-1.5, 0)$ and the high regions at the top and bottom edges. The RKS surface is smoother, while the MLP surface shows slight straight-edged facets because a ReLU network is piecewise linear.

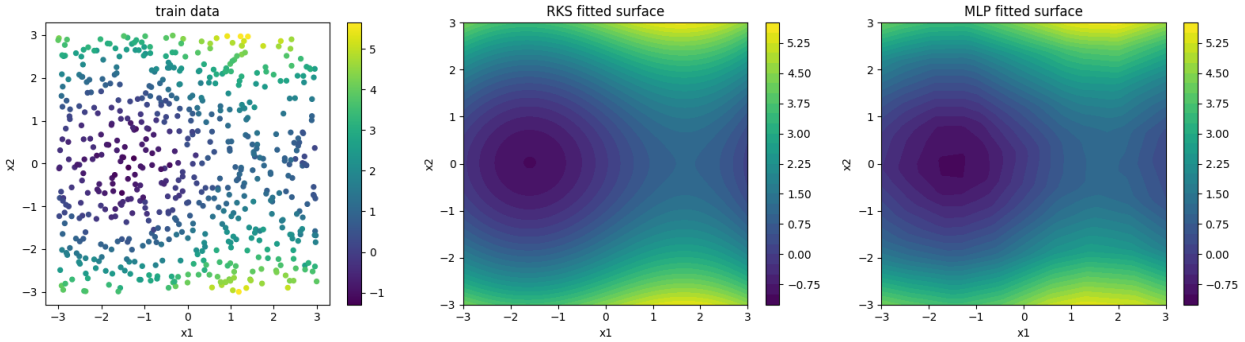


Figure 4: Training data (left), RKS surface (middle), MLP surface (right).